PRAHARI — how it's built

Team f20230436

This is the technical companion to `SUBMISSION.md`. It walks through how the system is put together, the calls we made along the way, and the numbers to back them up. If you only read one thing, read the two ideas in the next two sections. They are what the whole project rests on.

Idea 1: we watch the shock form instead of guessing when it arrives

A normal oil price-shock model treats the shock as a random event on a hidden clock. It has to estimate *when* a jump might happen and how often, and it can only do that after prices have already moved. That is the hard, uncertain part.

We removed that part. PRAHARI reads the news and the ship movements directly, so it sees the disruption while it is still forming. It never has to guess the timing. All it has to do is measure how large the shock is and how long it takes to settle. That is a smaller and more honest problem, and it is the reason the system can go from a detected signal to a full recommendation in milliseconds rather than after the fact.

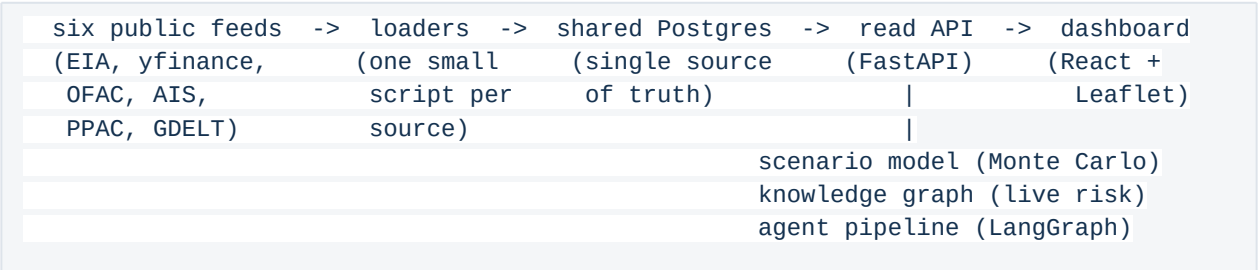
Idea 2: every number is computed, only the words come from an AI

We drew a hard line through the system. Everything quantitative, which is the risk score, the simulated price paths, the reserve-cover days, the travel times, the cost per day, is computed with plain code (SQL, numpy, and a graph). A language model is used in exactly two narrow places: to read messy news text and turn it into clean structured events, and, optionally, to write the final summary sentence.

This one rule buys three things:

- **You can audit it.** Every figure on screen can be recomputed by hand and traced back to the events that produced it. When you tell a government what a crisis costs, that matters.
- **It does not fall over.** The critical path has no live AI call in it, so there is nothing to hang or rate-limit at the worst moment. The summary line is a template. If the news-reading model is down, extraction falls back to rules and the pipeline still finishes.
- **It is fast.** No network round-trip to an AI sits between the signal and the answer.

The shape of the system



Nothing talks to the outside websites except the loaders. Everything else, the map, the model, the agents, reads from the same database, so no two parts can disagree about what the data says. That

single design choice removed a whole class of "your JSON does not match mine" bugs between four people working in parallel.

The data layer

Six feeds, each with its own small loader that pulls fresh data, cleans it, and upserts it into one table. All loaders are idempotent, so re-running anything is always safe. What is loaded right now:

- Daily Brent and WTI spot prices from the US government (EIA): 500 rows
- Intraday price ticks from Yahoo Finance: ~14,700 rows
- The US sanctions list (OFAC SDN): ~19,100 entries, ~1,500 of them vessels
- Live ship positions (AISStream): ~1,550 positions per snapshot
- India's monthly crude import bill (PPAC): 312 rows
- World news events scored into risk signals (GDELT): 480 events

A single controller reruns each feed on the cadence its source actually changes (prices every ~10 minutes, news every few hours, sanctions daily, imports monthly) and prunes only stale ship positions. It keeps price, import, and event history forever, because the scenario model calibrates on that history and the risk score already down-weights old news by age.

Two honest notes we put in front of judges rather than hide: the free ship-tracker has almost no coverage near the Persian Gulf, so the live ship dots sit on well-covered lanes to prove the pipeline is real; and the data refreshes on a schedule, it is not a live stream. Our rule throughout: real but slightly old beats made-up.

Turning news into a risk score

For a sea corridor, the risk score is a confidence- and recency-weighted average of event severity:

```
 risk = sum( severity/10 * confidence * decay ) / sum( confidence * decay )
decay = 0.5 ^ (days_old / 7)
```

An event from a week ago counts half as much as today's. It is a transparent heuristic, not a black box, which means we can explain any number on stage. When a score is backed by only a handful of events, the API flags it as low-evidence rather than pretending a 0.9 from three events is as solid as a 0.9 from three hundred. Corridor assignment is geofenced by the event's real coordinates, so a strike *at* Hormuz and a protest inland in Iran are no longer treated as the same thing.

The knowledge graph

A directed graph of India's real crude supply chain: 7 suppliers, their export ports, 5 sea chokepoints, Indian ports, and 5 refineries, wired the way the real infrastructure is, including the pipelines that bypass Hormuz (Saudi Arabia's line to Yanbu, the UAE's to Fujairah, Russia's to the Pacific). Iraq and Kuwait have no bypass, and the graph reproduces that true fact.

The skeleton is fixed, but the danger on each chokepoint is pulled live from the same news score at query time. So asking the graph "who can still reach Jamnagar today, by which route, in how many days" is a real path query over live risk, not a hardcoded answer. In one live run with Hormuz at 0.90

and the Red Sea also high, the graph returned: UAE via Fujairah in about 3 days, Nigeria via the Cape in ~26 days, the USA via the Cape in ~34 days, and correctly declared Iraq and Kuwait stuck. Nobody wrote that table. It fell out of the graph.

The scenario model

A mean-reverting jump-diffusion for Brent, run as 10,000 vectorised numpy paths in a few milliseconds. The jump is applied once, at the moment the signal fires, scaled by the corridor's risk score, and then decays back over a "days to reroute" window. From the price paths we derive a spread of outcomes: the likely and worst-case price jump, how far the reserve cover falls, the probability it drops below a safe floor, and the cost per day in rupees. We report ranges, not one fake-precise number, and we label the price-to-economy step honestly as a simplified elasticity rather than dressing it up as a full macro model.

The agent pipeline

The four steps (score the signal, simulate the scenario, rank the routes, write the summary) are wired into one linear chain with LangGraph. It runs synchronously and records each step's real wall-clock time, which is what the dashboard's latency read-out shows. There is an "inject signal" mode that runs the entire pipeline from a single cached real event and never touches the database, so the demo works even with no network. That is our on-stage safety net, and it is a real property of the code, not a mock.

Engineering we stand behind

- 36 automated tests plus a couple of integration suites, run on every push through GitHub Actions.
- Loaders validate their inputs: crude prices outside $1 to $500 a barrel are rejected as errors, and a truncated sanctions download fails loudly instead of quietly loading half a list.
- Downloads retry with backoff so a single network blip in the cloud refresher does not drop a feed.
- The read API degrades gracefully against an older database that predates a column, rather than crashing.
- Measured latency from signal to recommendation: about 20 milliseconds in our runs.
- Everything runs on free tiers. No paid infrastructure.

What we cut on purpose, and why

Being deliberate about scope is a feature, not an apology. These were choices:

We cut	Why
AI-generated summary on the critical path	A template has zero latency and zero failure risk right before the punchline. A judge cannot tell them apart.
Two price archetypes	One calibrated, severity-scaled jump does the same job with half the code and half the surface to defend live.

Real-time streaming (SSE, Redis)	A scripted reveal using the real recorded step timings gives the same visual effect without the streaming infrastructure risk.
Docker and NGINX	We present from a laptop. Containerising a single demo run buys nothing.
A separate reserve agent and a CVaR optimiser	Stretch goals. We said them in the pitch instead of half-building them.

Tech stack

Python and FastAPI for the service, Postgres on Neon for the shared database, numpy for the Monte Carlo, networkx for the knowledge graph, LangGraph for the agent chain, React with Vite and Leaflet for the dashboard, GitHub Actions for CI and the scheduled data refresh. A language model (via an OpenAI-compatible API) reads the news; nothing else in the system depends on it.